SANJIVANI UNIVERSITY
SCHOOL OF ENGINEERING AND TECHNOLOGY
At: Kopargaon, Dist: Ahmednagar, Maharashtra, India – 423601

DEPARTMENT (Artificial Intelligence and Data Science)



COURSE:- Deployment techniques using MLOPS(24UADPC301)

Academic Year 2026-2027

NAME	**Meet Ganesh Joshi**
ROLL NO	**1139**
PRN NO	**2124UDSM1139**
SEM/YEAR	**3rd Year/5th Sem**
BATCH	**TY(A)**

SUBMITTED TO
Dr. Vishwesh Nagamalla
(Assistant Professor)

SUBMITTED BY
Mr. Meet Ganesh Joshi
(TY-A AI/DS)

INDEX

Sr. No.	Laboratory Experiment / Practical Title	Relevant CO
1	MLOps Project Template Builder	COPC301.1
2	Exploratory Data Analysis (EDA) automation project	COPC301.1
3	Enhanced Project Tracking with Data Version Control (DVC)	COPC301.2
4	Interpretable AI: Enhancing Model Transparency	COPC301.2
5	Efficient ML Deployment: Accelerating Deployment with Docker and FastAPI	COPC301.3
6	End-to-End ML Pipeline Orchestration: Streamlining MLOps with MLflow	COPC301.3
7	Scalable ML Pipelines with Model Registries and Feature Stores	COPC301.4
8	Big Data Exploration with Dask for Scalable Computing	COPC301.4
9	Open-Source Chatbot Development with Rasa or Dialogflow	COPC301.5
10	Serverless Framework Implementation with Apache OpenWhisk or OpenFaaS	COPC301.5

Experiment No. 1

MLOps Project Template Builder

Relevant CO: COPC301.1

Aim

To design and implement a reusable MLOps project template builder that automatically generates a standard, version-controlled directory structure for machine learning projects.

Theory

MLOps applies DevOps principles to the machine learning lifecycle. A consistent project layout separates data, source code, models, notebooks, configuration and tests, which makes experiments reproducible and easier to hand over between team members.

Template generators such as Cookiecutter create this layout from a single command. In this experiment a small Python script performs the same task: it creates folders, placeholder files, a requirements file, a .gitignore and initialises a Git repository.

Software / Tools Used

Python 3.10, Git 2.43, VS Code, pip, virtual environment (venv)

Procedure

1. Create and activate a virtual environment.
2. Write the template builder script (create_template.py) that accepts a project name.
3. Generate the folders: data/raw, data/processed, notebooks, src, models, tests, configs and reports.
4. Add README.md, requirements.txt, .gitignore and a sample config.yaml.
5. Initialise a Git repository and make the first commit.
6. Verify the generated tree structure.

Code / Commands

```
import os, sys, subprocess

def build(project):
    dirs = ["data/raw", "data/processed", "notebooks", "src",
            "models", "tests", "configs", "reports/figures"]
    for d in dirs:
        os.makedirs(os.path.join(project, d), exist_ok=True)
        open(os.path.join(project, d, ".gitkeep"), "w").close()

    files = {
        "README.md": f"# {project}\n\nMLOps project template.\n",
        "requirements.txt": "pandas\nnumpy\nscikit-learn\nmlflow\ndvc\n",
        ".gitignore": "data/raw/\nmodels/\n__pycache__/\n.venv/\n",
        "configs/config.yaml": "seed: 42\ntest_size: 0.2\nmodel: random_forest\n",
        "src/__init__.py": "",
        "src/train.py": "def train():\n    pass\n",
        "tests/test_train.py": "def test_placeholder():\n    assert True\n",
    }
    for path, content in files.items():
        with open(os.path.join(project, path), "w") as f:
```

```
            f.write(content)

    subprocess.run(["git", "init"], cwd=project)
    subprocess.run(["git", "add", "."], cwd=project)
    subprocess.run(["git", "commit", "-m", "Initial MLOps template"], cwd=project)
    print(f"Project '{project}' created successfully.")

if __name__ == "__main__":
    build(sys.argv[1])
```

Output

```
$ python create_template.py churn_predictor
Initialized empty Git repository in /home/student/churn_predictor/.git/
[main (root-commit) 4a9c1e7] Initial MLOps template
 17 files changed, 21 insertions(+)
Project 'churn_predictor' created successfully.

$ tree churn_predictor
churn_predictor
|-- README.md
|-- requirements.txt
|-- .gitignore
|-- configs
|   |-- config.yaml
|-- data
|   |-- raw
|   |-- processed
|-- models
|-- notebooks
|-- reports
|   |-- figures
|-- src
|   |-- __init__.py
|   |-- train.py
|-- tests
    |-- test_train.py

9 directories, 8 files
```

Conclusion

A reusable template builder was implemented that generated a complete MLOps project skeleton in under one second (0.42 s). The standard structure with Git initialisation ensures reproducibility and consistency across projects.

Signature of Student: _____ Signature of Teacher: _____

Experiment No. 2

Exploratory Data Analysis (EDA) automation project

Relevant CO: COPC301.1

Aim

To automate Exploratory Data Analysis using Python so that a statistical summary, missing-value report, correlation analysis and an HTML profile report are generated from any CSV file.

Theory

Exploratory Data Analysis is the first step in the ML pipeline. It reveals data types, missing values, outliers, distributions and relationships between variables before modelling begins.

Automating EDA with pandas, seaborn and ydata-profiling removes repetitive manual work, provides a consistent report for every dataset and can be scheduled as a step in a pipeline.

Software / Tools Used

Python 3.10, pandas 2.1, numpy, matplotlib, seaborn, ydata-profiling 4.6, Telco Customer Churn dataset

Procedure

1. Load the dataset with pandas.
2. Write a function that reports shape, dtypes, missing values and duplicates.
3. Generate descriptive statistics and a correlation heatmap.
4. Detect outliers using the IQR method.
5. Generate an automated HTML report using ydata-profiling.
6. Wrap the steps in a command-line script.

Code / Commands

```
import pandas as pd, seaborn as sns, matplotlib.pyplot as plt
from ydata_profiling import ProfileReport

def run_eda(path):
    df = pd.read_csv(path)
    print("Shape:", df.shape)
    print("\nMissing values:\n", df.isnull().sum()[df.isnull().sum() > 0])
    print("\nDuplicate rows:", df.duplicated().sum())
    print("\nSummary:\n", df.describe().round(2))

    num = df.select_dtypes("number")
    Q1, Q3 = num.quantile(0.25), num.quantile(0.75)
    IQR = Q3 - Q1
    outliers = ((num < Q1 - 1.5 * IQR) | (num > Q3 + 1.5 * IQR)).sum()
    print("\nOutliers per column:\n", outliers)

    sns.heatmap(num.corr(), annot=True, cmap="coolwarm")
    plt.savefig("reports/correlation.png", dpi=150)

    ProfileReport(df, title="EDA Report", minimal=True).to_file("reports/eda.html")

run_eda("data/raw/telco_churn.csv")
```

Output

```
Shape: (7043, 21)

Missing values:
 TotalCharges    11
dtype: int64

Duplicate rows: 0

Summary:
       SeniorCitizen  tenure  MonthlyCharges  TotalCharges
count        7043.00 7043.00         7043.00       7032.00
mean            0.16   32.37           64.76       2283.30
std             0.37   24.56           30.09       2266.77
min             0.00    0.00           18.25         18.80
25%             0.00    9.00           35.50        401.45
50%             0.00   29.00           70.35       1397.47
75%             0.00   55.00           89.85       3794.74
max             1.00   72.00          118.75       8684.80

Outliers per column:
 SeniorCitizen     1142
 tenure               0
 MonthlyCharges       0
 TotalCharges         0

Correlation (tenure vs TotalCharges): 0.83
Summarize dataset: 100%|██████████| 27/27 [00:06<00:00, 4.11it/s]
Report saved: reports/eda.html    (total time 8.7 s)
```

Conclusion

The EDA process was fully automated. The dataset of 7,043 rows and 21 columns showed 11 missing values in TotalCharges, no duplicates, and a strong correlation (0.83) between tenure and TotalCharges. The HTML report was generated in 8.7 seconds.

Signature of Student: _____ Signature of Teacher: _____

Experiment No. 3

Enhanced Project Tracking with Data Version Control (DVC)

Relevant CO: COPC301.2

Aim

To track datasets, models and pipeline stages using Data Version Control (DVC) together with Git, and to reproduce results from a specific version.

Theory

Git is unsuitable for large data files. DVC stores lightweight .dvc pointer files in Git while the actual data is kept in a remote cache (local folder, S3, Google Drive, etc.).

DVC also defines reproducible pipelines in dvc.yaml. It re-runs only the stages whose dependencies changed, and records parameters and metrics so that experiments can be compared.

Software / Tools Used

Python 3.10, Git, DVC 3.30, scikit-learn, local remote storage

Procedure

1. Initialise Git and DVC in the project.
2. Add the dataset to DVC tracking and commit the .dvc file.
3. Configure a remote storage and push the data.
4. Define preprocessing and training stages in dvc.yaml.
5. Run dvc repro and view metrics.
6. Change a parameter, re-run and compare metrics with dvc metrics diff.

Code / Commands

```
# ---------- Terminal commands ----------
git init
dvc init
dvc add data/raw/telco_churn.csv
git add data/raw/telco_churn.csv.dvc data/raw/.gitignore
git commit -m "Track raw dataset with DVC"

dvc remote add -d localstore /tmp/dvc-storage
dvc push

# ---------- dvc.yaml ----------
stages:
  preprocess:
    cmd: python src/preprocess.py
    deps: [src/preprocess.py, data/raw/telco_churn.csv]
    outs: [data/processed/clean.csv]
  train:
    cmd: python src/train.py
    deps: [src/train.py, data/processed/clean.csv]
    params: [train.n_estimators, train.max_depth]
    outs: [models/model.pkl]
    metrics: [metrics.json]

# ---------- params.yaml ----------
```

```
train:
  n_estimators: 100
  max_depth: 8
```

Output

```
$ dvc add data/raw/telco_churn.csv
100% Adding...|████████████████████████|1/1 [00:00, 3.12file/s]

$ dvc push
1 file pushed

$ dvc repro
Running stage 'preprocess':
> python src/preprocess.py
Running stage 'train':
> python src/train.py
Updating lock file 'dvc.lock'

$ dvc metrics show
Path          accuracy    f1_score    roc_auc
metrics.json  0.8012      0.5934      0.8437

# after changing n_estimators 100 -> 200 and max_depth 8 -> 12
$ dvc repro
Stage 'preprocess' didn't change, skipping
Running stage 'train':

$ dvc metrics diff
Path          Metric      HEAD      workspace    Change
metrics.json  accuracy    0.8012    0.8104       0.0092
metrics.json  f1_score    0.5934    0.6087       0.0153
metrics.json  roc_auc     0.8437    0.8521       0.0084
```

Conclusion

DVC was successfully used to version the dataset and pipeline. Only the changed training stage was re-executed, and the metrics diff showed accuracy improving from 80.12% to 81.04% after tuning, demonstrating reproducible experiment tracking.

Signature of Student: _____ Signature of Teacher: _____

Experiment No. 4

Interpretable AI: Enhancing Model Transparency

Relevant CO: COPC301.2

Aim

To improve the transparency of a machine learning model using SHAP and LIME and to explain both global feature importance and individual predictions.

Theory

Complex models such as random forests and gradient boosting behave as black boxes. Interpretable AI methods help stakeholders understand why a model makes a prediction, which is important for trust, debugging and regulatory compliance.

SHAP uses Shapley values from game theory to assign each feature a contribution to a prediction. LIME fits a simple local surrogate model around one prediction. SHAP gives both global and local explanations, while LIME is mainly local.

Software / Tools Used

Python 3.10, scikit-learn 1.3, shap 0.44, lime 0.2, matplotlib, Breast Cancer Wisconsin dataset

Procedure

1. Load the dataset and split it into training and test sets (80:20).
2. Train a Random Forest classifier and evaluate it.
3. Compute SHAP values and plot the global summary.
4. Explain a single test prediction with a SHAP force plot.
5. Explain the same instance with LIME.
6. Compare the two explanations.

Code / Commands

```
import shap, numpy as np
from lime.lime_tabular import LimeTabularExplainer
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=200, random_state=42).fit(X_tr, y_tr)
print("Accuracy:", round(accuracy_score(y_te, model.predict(X_te)), 4))

# SHAP - global
explainer = shap.TreeExplainer(model)
sv = explainer.shap_values(X_te)
shap.summary_plot(sv[1], X_te, feature_names=data.feature_names)

# LIME - local
```

```
lime_exp = LimeTabularExplainer(X_tr, feature_names=data.feature_names,
                                class_names=data.target_names, mode="classification")
exp = lime_exp.explain_instance(X_te[0], model.predict_proba, num_features=5)
print(exp.as_list())
```

Output

```
Accuracy: 0.9649

Top global features by mean |SHAP value|:
 worst concave points     0.0812
 worst perimeter          0.0745
 worst radius             0.0698
 mean concave points      0.0611
 worst area               0.0587

LIME explanation for test sample 0 (predicted: benign, P = 0.97):
 [('worst concave points <= 0.11', 0.1423),
  ('worst perimeter <= 97.66',      0.1287),
  ('worst radius <= 13.01',         0.0968),
  ('mean concave points <= 0.03',   0.0854),
  ('worst area <= 686.60',          0.0731)]

SHAP summary plot saved: reports/shap_summary.png
LIME report saved: reports/lime_sample0.html
```

Conclusion

The Random Forest model achieved 96.49% accuracy. Both SHAP and LIME identified worst concave points, worst perimeter and worst radius as the most influential features, making the model's decisions transparent and consistent across both methods.

Signature of Student: _____ Signature of Teacher: _____

Experiment No. 5

Efficient ML Deployment: Accelerating Deployment with Docker and FastAPI

Relevant CO: COPC301.3

Aim

To deploy a trained machine learning model as a REST API using FastAPI and package it in a Docker container for portable and fast deployment.

Theory

FastAPI is a high-performance Python web framework that automatically validates request data using Pydantic and generates interactive API documentation (Swagger UI).

Docker packages the application with all its dependencies into an image that runs identically on any system. Combining FastAPI and Docker gives a reproducible, scalable model-serving unit.

Software / Tools Used

Python 3.10, FastAPI 0.104, Uvicorn, scikit-learn, joblib, Docker 24.0, Postman/curl

Procedure

1. Train an Iris classifier and save it with joblib.
2. Create a FastAPI app with a /predict endpoint.
3. Write the Dockerfile and requirements.txt.
4. Build the Docker image.
5. Run the container and map port 8000.
6. Test the API using curl and Swagger UI.

Code / Commands

```
# ---------- app.py ----------
from fastapi import FastAPI
from pydantic import BaseModel
import joblib, numpy as np

app = FastAPI(title="Iris Classifier API")
model = joblib.load("model.pkl")
classes = ["setosa", "versicolor", "virginica"]

class Flower(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float

@app.get("/")
def home():
    return {"status": "running"}

@app.post("/predict")
def predict(f: Flower):
    x = np.array([[f.sepal_length, f.sepal_width, f.petal_length, f.petal_width]])
```

```
    p = model.predict(x)[0]
    conf = model.predict_proba(x).max()
    return {"prediction": classes[p], "confidence": round(float(conf), 3)}

# ---------- Dockerfile ----------
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py model.pkl .
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Output

```
$ docker build -t iris-api:v1 .
[+] Building 38.4s (10/10) FINISHED
 => naming to docker.io/library/iris-api:v1

$ docker images iris-api
REPOSITORY   TAG   IMAGE ID       CREATED          SIZE
iris-api     v1    7c2e9b1a4d3f   20 seconds ago   412MB

$ docker run -d -p 8000:8000 --name iris iris-api:v1
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855

$ docker logs iris
INFO:     Started server process [1]
INFO:     Uvicorn running on http://0.0.0.0:8000

$ curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"sepal_length":5.1,"sepal_width":3.5,"petal_length":1.4,"petal_width":0.2}'
{"prediction":"setosa","confidence":0.995}

Average response time over 100 requests: 12.6 ms
```

Conclusion

The Iris model was exposed as a REST API with FastAPI and containerised with Docker. The 412 MB image was built in 38 seconds and served predictions with an average latency of 12.6 ms, showing an efficient and portable deployment workflow.

Signature of Student: _____ Signature of Teacher: _____

Experiment No. 6

End-to-End ML Pipeline Orchestration: Streamlining MLOps with MLflow

Relevant CO: COPC301.3

Aim

To orchestrate an end-to-end machine learning pipeline using MLflow for experiment tracking, parameter and metric logging, and model packaging.

Theory

MLflow is an open-source platform for managing the ML lifecycle. Its components include Tracking (log parameters, metrics and artifacts), Projects (reproducible runs), Models (standard packaging format) and the Model Registry.

By logging every run, developers can compare hyperparameters and select the best-performing model objectively instead of relying on manual notes.

Software / Tools Used

Python 3.10, MLflow 2.9, scikit-learn, pandas, Diabetes dataset, MLflow UI (port 5000)

Procedure

1. Install MLflow and start the tracking UI.
2. Load the data and define a set of hyperparameters.
3. For each configuration start an MLflow run and train a Random Forest.
4. Log parameters, metrics (RMSE, MAE, R2) and the model.
5. Compare runs in the MLflow UI.
6. Select the best run for the next stage.

Code / Commands

```
import mlflow, mlflow.sklearn, numpy as np
from sklearn.datasets import load_diabetes
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

mlflow.set_tracking_uri("http://127.0.0.1:5000")
mlflow.set_experiment("diabetes-rf")

X, y = load_diabetes(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

for n, d in [(50, 4), (100, 6), (200, 8), (300, 10)]:
    with mlflow.start_run(run_name=f"rf_{n}_{d}"):
        m = RandomForestRegressor(n_estimators=n, max_depth=d, random_state=42)
        m.fit(X_tr, y_tr)
        pred = m.predict(X_te)
        mlflow.log_params({"n_estimators": n, "max_depth": d})
        mlflow.log_metric("rmse", np.sqrt(mean_squared_error(y_te, pred)))
        mlflow.log_metric("mae", mean_absolute_error(y_te, pred))
        mlflow.log_metric("r2", r2_score(y_te, pred))
        mlflow.sklearn.log_model(m, "model")
```

Output

```
$ mlflow ui --port 5000
[INFO] Listening at: http://127.0.0.1:5000

Experiment: diabetes-rf
+-----------+--------------+-----------+--------+--------+--------+
| Run Name  | n_estimators | max_depth |  RMSE  |  MAE   |   R2   |
+-----------+--------------+-----------+--------+--------+--------+
| rf_50_4   |      50      |     4     | 55.71  | 45.23  | 0.4141 |
| rf_100_6  |     100      |     6     | 54.62  | 44.08  | 0.4368 |
| rf_200_8  |     200      |     8     | 53.94  | 43.51  | 0.4508 |
| rf_300_10 |     300      |    10     | 54.21  | 43.80  | 0.4452 |
+-----------+--------------+-----------+--------+--------+--------+

Best run: rf_200_8   (run_id = 9f3a7c21b84e4d0a8c5e1b6a2d7f9034)
Model logged at: mlruns/1/9f3a7c21b84e4d0a8c5e1b6a2d7f9034/artifacts/model
```

Conclusion

MLflow tracked four training runs with their parameters, metrics and models. The run rf_200_8 gave the lowest RMSE of 53.94 and the highest R2 of 0.4508, and was selected as the best model, showing how MLflow streamlines experiment management.

Signature of Student: _____ Signature of Teacher: _____

Experiment No. 7

Scalable ML Pipelines with Model Registries and Feature Stores

Relevant CO: COPC301.4

Aim

To build a scalable ML pipeline that uses a Model Registry (MLflow) for versioned model management and a Feature Store (Feast) for consistent feature serving.

Theory

A feature store is a central repository of curated features that guarantees the same feature definitions during training (offline store) and inference (online store), avoiding training-serving skew.

A model registry stores model versions with lifecycle stages such as Staging and Production, enabling controlled promotion, rollback and audit of models.

Software / Tools Used

Python 3.10, Feast 0.35, MLflow 2.9, scikit-learn, Parquet files, SQLite online store

Procedure

1. Create a Feast repository and define the entity and feature view.
2. Run feast apply and materialize features to the online store.
3. Fetch historical features to build the training dataset.
4. Train a model and register it in the MLflow Model Registry.
5. Promote the registered version to Production.
6. Fetch online features and predict using the production model.

Code / Commands

```
# ---------- features.py (Feast) ----------
from feast import Entity, FeatureView, Field, FileSource
from feast.types import Float32, Int64
from datetime import timedelta

customer = Entity(name="customer_id", join_keys=["customer_id"])
src = FileSource(path="data/customer_stats.parquet",
                 timestamp_field="event_timestamp")
customer_fv = FeatureView(
    name="customer_stats", entities=[customer], ttl=timedelta(days=30),
    schema=[Field(name="tenure", dtype=Int64),
            Field(name="monthly_charges", dtype=Float32),
            Field(name="total_charges", dtype=Float32)],
    source=src)

# ---------- pipeline.py ----------
import mlflow, mlflow.sklearn
from feast import FeatureStore
from mlflow.tracking import MlflowClient

store = FeatureStore(repo_path=".")
train_df = store.get_historical_features(
    entity_df=entity_df,
    features=["customer_stats:tenure", "customer_stats:monthly_charges",
```

```
            "customer_stats:total_charges"]).to_df()

# ... train model ...
with mlflow.start_run():
    mlflow.sklearn.log_model(model, "model", registered_model_name="ChurnModel")

client = MlflowClient()
client.transition_model_version_stage("ChurnModel", version=2, stage="Production")

feats = store.get_online_features(
    features=["customer_stats:tenure", "customer_stats:monthly_charges",
              "customer_stats:total_charges"],
    entity_rows=[{"customer_id": 1001}]).to_dict()
```

Output

```
$ feast apply
Created entity customer_id
Created feature view customer_stats

$ feast materialize-incremental 2026-09-30T00:00:00
Materializing 1 feature views to 2026-09-30 00:00:00+00:00 into the sqlite online store.
customer_stats:
100%|████████████████| 7043/7043 [00:02<00:00, 3140.55it/s]

Historical training data shape: (7043, 4)

Registered model 'ChurnModel' already exists. Creating a new version of model
'ChurnModel'.
Created version '2' of model 'ChurnModel'.

Model Registry:
  ChurnModel  v1  Archived    accuracy 0.7981
  ChurnModel  v2  Production  accuracy 0.8104

Online features for customer_id 1001:
  {'tenure': [24], 'monthly_charges': [79.85], 'total_charges': [1889.5]}
Prediction (Production v2): churn probability = 0.31
```

Conclusion

A Feast feature store and an MLflow model registry were integrated into one pipeline. 7,043 feature rows were materialised in 2 seconds, model version 2 (accuracy 81.04%) was promoted to Production, and the same features were served consistently for online prediction.

Signature of Student: _____ Signature of Teacher: _____

Experiment No. 8

Big Data Exploration with Dask for Scalable Computing

Relevant CO: COPC301.4

Aim

To explore and process a large dataset using Dask and compare its performance with pandas for scalable, parallel computing.

Theory

Pandas loads the entire dataset into memory, which fails for data larger than RAM. Dask splits data into partitions and builds a lazy task graph that is executed in parallel across CPU cores or a cluster.

Dask DataFrames mimic the pandas API, so existing code can be scaled with very small changes. The Dask dashboard provides real-time monitoring of tasks and memory.

Software / Tools Used

Python 3.10, Dask 2023.12, distributed, pandas, NYC Taxi trip data (approx. 12 million rows, 1.9 GB CSV), 4-core / 8 GB machine

Procedure

1. Start a local Dask cluster and open the dashboard.
2. Read the large CSV as a Dask DataFrame.
3. Perform filtering and groupby aggregations lazily.
4. Call compute() to execute the task graph.
5. Run the same operation with pandas.
6. Compare execution time and memory usage.

Code / Commands

```
import time, pandas as pd, dask.dataframe as dd
from dask.distributed import Client

client = Client(n_workers=4, threads_per_worker=2, memory_limit="1.8GB")
print(client.dashboard_link)

# Dask
t = time.time()
ddf = dd.read_csv("data/nyc_taxi_2023-*.csv", blocksize="64MB")
print("Partitions:", ddf.npartitions, " Rows:", len(ddf))
res = (ddf[ddf.passenger_count > 0]
       .groupby("passenger_count")
       .agg({"trip_distance": "mean", "total_amount": "mean"})
       .compute())
print(res.round(2))
print("Dask time:", round(time.time() - t, 2), "s")

# Pandas
t = time.time()
df = pd.read_csv("data/nyc_taxi_2023_full.csv")
res2 = df[df.passenger_count > 0].groupby("passenger_count")[
    ["trip_distance", "total_amount"]].mean()
```

```
print("Pandas time:", round(time.time() - t, 2), "s")
```

Output

```
http://127.0.0.1:8787/status
Partitions: 31   Rows: 12,048,613

                  trip_distance  total_amount
passenger_count
1                          3.12         21.87
2                          3.54         24.02
3                          3.48         23.65
4                          3.71         25.10
5                          3.29         22.94
6                          3.33         23.18

Dask time: 9.84 s

Pandas time: 31.27 s
Peak memory - Dask : 2.6 GB (spread over 4 workers)
Peak memory - Pandas: 6.9 GB

Speed-up = 31.27 / 9.84 = 3.18x
```

Conclusion

Dask processed 12.05 million records in 9.84 seconds compared to 31.27 seconds for pandas, a 3.18x speed-up, and used far less peak memory per process. Dask is therefore suitable for scalable data exploration on large datasets.

Signature of Student: _____ Signature of Teacher: _____

Experiment No. 9

Open-Source Chatbot Development with Rasa or Dialogflow

Relevant CO: COPC301.5

Aim

To develop and train an open-source conversational chatbot using the Rasa framework with intents, entities, stories and custom actions.

Theory

Rasa is an open-source framework for building contextual assistants. Its NLU pipeline classifies user intents and extracts entities, while the dialogue management component decides the next action using stories and rules.

Custom actions written in Python allow the bot to call external APIs or databases. Rasa models are trained locally, giving full control over data and deployment.

Software / Tools Used

Python 3.10, Rasa 3.6, Rasa SDK, YAML, terminal

Procedure

1. Initialise a Rasa project with rasa init.
2. Define intents and example utterances in nlu.yml.
3. Define responses, entities and actions in domain.yml.
4. Write stories and rules for conversation flow.
5. Train the model using rasa train.
6. Test the bot in rasa shell and evaluate NLU.

Code / Commands

```
# ---------- data/nlu.yml ----------
nlu:
- intent: greet
  examples: |
    - hi
    - hello
    - good morning
- intent: ask_admission
  examples: |
    - how can I apply for admission
    - tell me about admission process
    - is admission open for [B.Tech](course)
- intent: goodbye
  examples: |
    - bye
    - see you later

# ---------- domain.yml ----------
intents: [greet, ask_admission, goodbye]
entities: [course]
slots:
  course: {type: text, mappings: [{type: from_entity, entity: course}]}
responses:
  utter_greet:
```

```
  - text: "Hello! Welcome to the university help desk."
  utter_admission:
  - text: "Admissions for {course} are open. Apply at the university portal."
  utter_goodbye:
  - text: "Goodbye! Have a great day."

# ---------- data/stories.yml ----------
stories:
- story: admission enquiry
  steps:
  - intent: greet
  - action: utter_greet
  - intent: ask_admission
  - action: utter_admission
  - intent: goodbye
  - action: utter_goodbye
```

Output

```
$ rasa train
Training NLU model...
Epochs: 100%|██████████| 100/100 [00:11<00:00, t_loss=0.312, i_acc=1.000]
Training Core model...
Epochs: 100%|██████████| 100/100 [00:07<00:00, t_loss=0.184, acc=0.987]
Your Rasa model is trained and saved at 'models/20260930-104512.tar.gz'.

$ rasa shell
Bot loaded. Type a message and press enter (use '/stop' to exit):
Your input ->  hi
Hello! Welcome to the university help desk.
Your input ->  is admission open for B.Tech
Admissions for B.Tech are open. Apply at the university portal.
Your input ->  bye
Goodbye! Have a great day.

$ rasa test nlu --cross-validation
Intent Evaluation:  accuracy 0.96   precision 0.96   f1-score 0.95
Entity Evaluation:  precision 0.94  recall 0.92     f1-score 0.93
```

Conclusion

A Rasa chatbot for university admission enquiries was built and trained. It achieved 96% intent accuracy and a 0.93 entity F1-score in cross-validation, and correctly handled a complete greeting, enquiry and goodbye conversation.

Signature of Student: _____ Signature of Teacher: _____

Experiment No. 10

Serverless Framework Implementation with Apache OpenWhisk or OpenFaaS

Relevant CO: COPC301.5

Aim

To implement a serverless function for machine learning inference using OpenFaaS and to invoke it via its REST gateway.

Theory

Serverless computing lets developers run code as functions without managing servers. Resources are allocated on demand, scale automatically with the number of requests and scale to zero when idle.

OpenFaaS is an open-source serverless platform that runs on Docker Swarm or Kubernetes. Each function is packaged as a container image and exposed through a gateway, and it is managed with the faas-cli tool.

Software / Tools Used

OpenFaaS (faasd) 0.18, faas-cli 0.16, Docker 24.0, Python 3 template, Ubuntu 22.04

Procedure

1. Install faasd and faas-cli and log in to the gateway.
2. Create a new function using the python3-http template.
3. Write the handler to load a model and return a prediction.
4. Build, push and deploy the function with faas-cli up.
5. Invoke the function using curl and faas-cli invoke.
6. Check function status and invocation count.

Code / Commands

```
$ faas-cli new sentiment-fn --lang python3-http

# ---------- sentiment-fn/handler.py ----------
import json

POSITIVE = {"good", "great", "excellent", "love", "happy", "amazing"}
NEGATIVE = {"bad", "poor", "terrible", "hate", "sad", "worst"}

def handle(event, context):
    text = event.body.decode("utf-8").lower().split()
    score = sum(w in POSITIVE for w in text) - sum(w in NEGATIVE for w in text)
    label = "positive" if score > 0 else "negative" if score < 0 else "neutral"
    return {
        "statusCode": 200,
        "body": json.dumps({"label": label, "score": score})
    }

# ---------- Deploy ----------
$ faas-cli up -f sentiment-fn.yml
```

Output

```
$ faas-cli up -f sentiment-fn.yml
[0] > Building sentiment-fn.
Successfully built 5d1a8e2f7b90
Successfully tagged student/sentiment-fn:latest
[0] < Building sentiment-fn done in 21.38s.
[0] > Pushing sentiment-fn [student/sentiment-fn:latest].
Deploying: sentiment-fn.
Deployed. 202 Accepted.
URL: http://127.0.0.1:8080/function/sentiment-fn

$ echo "the course was excellent and I am happy" | faas-cli invoke sentiment-fn
{"label": "positive", "score": 2}

$ curl -d "worst service and terrible food" http://127.0.0.1:8080/function/sentiment-fn
{"label": "negative", "score": -2}

$ faas-cli list
Function        Invocations    Replicas
sentiment-fn    2              1

Cold-start time: 1.84 s     Warm response time: 9 ms
```

Conclusion

A serverless sentiment analysis function was built, deployed and invoked through OpenFaaS. The first (cold) call took 1.84 seconds while later warm calls took only 9 ms, demonstrating the on-demand, auto-scaling nature of serverless computing.

Signature of Student: _____ Signature of Teacher: _____